# 实验五代码

## test.l

```
%{
#include "test5-1.tab.h"

int yywrap(void){
  return 1;
}
%}

ws          [ \t]

%%

"true"      { return TRUE; }
"false"     { return FALSE; }
"or"        { return OR; }
"and"       { return AND; }
"not"       { return NOT; }
"("         { return LPAREN; }
")"         { return RPAREN; }
ws          { ; }
"\n"        { return ENTER; }
.                   {;}

```

## test.y

```
%{
    #include <stdio.h>
    int yylex();
    #define YYSTYPE double /* 将Yacc栈定义为double类型 */
    int yyerror(char *);   /* 不声明该函数将出现warning */
%}

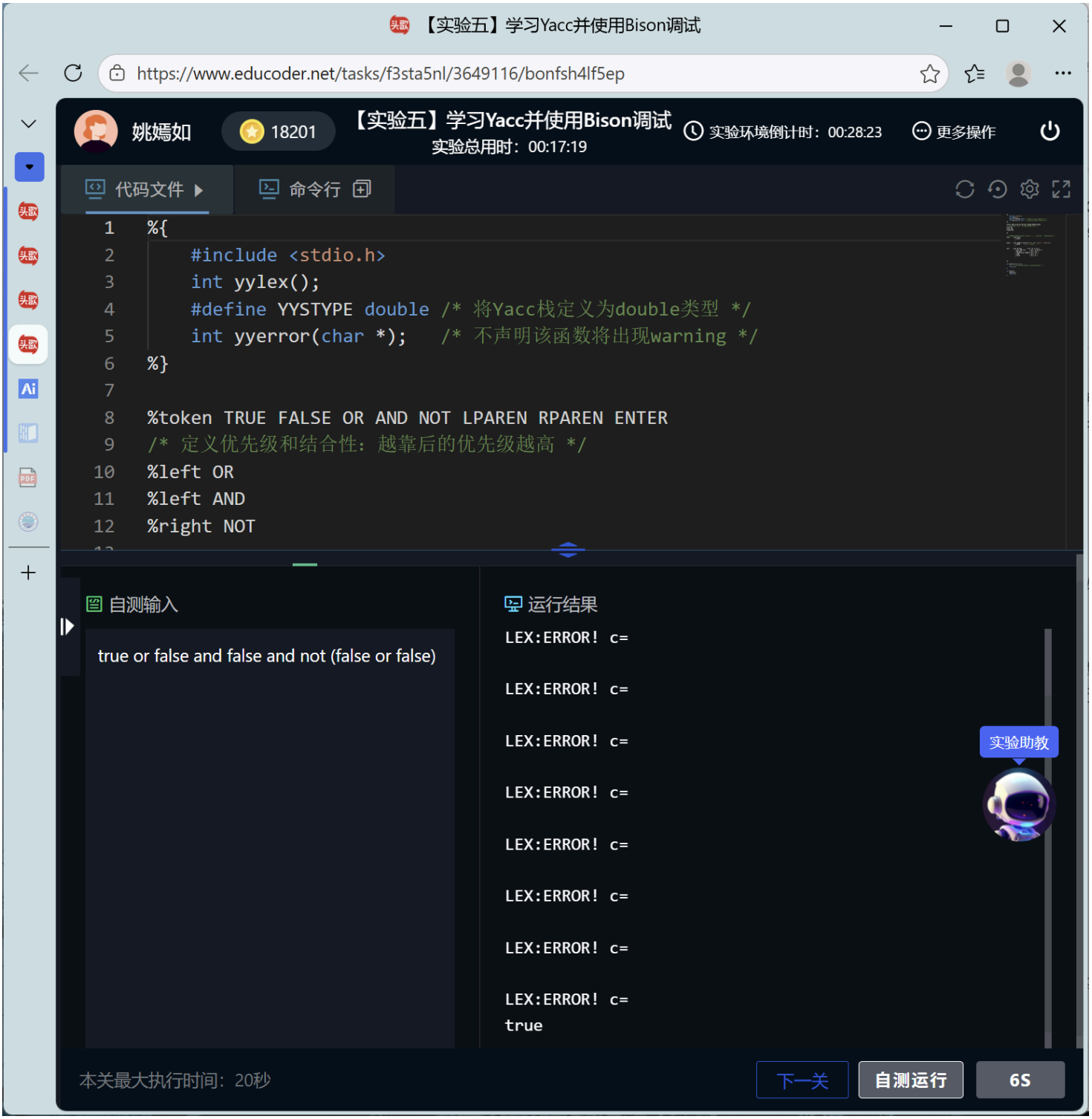
%token TRUE FALSE OR AND NOT LPAREN RPAREN ENTER
/* 定义优先级和结合性：越靠后的优先级越高 */
%left OR
%left AND
%right NOT

%%
```

```
15
16  /* 这样写prog可以让分析器每次读入一行进行分析，下一行重新分析expr */
17 prog    : prog exprp
18              | exprp
19              ;
20
21 exprp   : expr ENTER { printf("%s\n", $1 ? "true" : "false");}
22              | ENTER      { ; } /* 允许空行 */
23              ;
24
25 expr    : expr OR expr       { $$ = $1 || $3; }
26              | expr AND expr      { $$ = $1 && $3; }
27              | NOT expr           { $$ = !$2; }
28              | LPAREN expr RPAREN { $$ = $2; }
29              | TRUE               { $$ = 1; }
30              | FALSE              { $$ = 0; }
31              ;
32
33
34 %%
35
36 int yyerror(char *s) {
37     /* 按照惯例，语法错误时不输出额外信息或仅返回 */
38     return 0;
39 }
40
41 int main(){
42     yyparse();
43     return 0;
44 }
45
```

## 实验五 调试问题

1、在自此运行时，发现结果不仅输出了true，前面还有一堆报错输出




原因：查看代码后，发现实验初始代码中，没有正确处理定义之外的字符，导致每遇到一个无法匹配的字符就打印了一次错误信息。且根据输出，可以发现该字符是空格，只要碰到空格就会输出一次。

```
.                {printf("\nLEX:ERROR! c=%s\n", yytext);}
```

👉 修改代码如下：

直接跳过，不做任何动作。

```
.                {;}
```

# 实验六 代码

## test5-2.l

```
%{
#include "test5-2.tab.h"
#include <string.h>

int yywrap(void){
  return 1;
}
%}

ws          [ \t]

%%

"true"    {return TRUE_VAL;}
"false"     {return FALSE_VAL;}
"or"            {return OR;}
"and"           {return AND;}
"not"           {return NOT;}
"("           {return LPAREN;}
")"         {return RPAREN;}
{ws}      {;}
"\n"          {return ENTER;}
.                 {}

```

## test5-2.y

```
    %{
    #include <stdio.h>
    #include <stdlib.h>
    int yylex();
    #define YYSTYPE int /* 布尔运算，将Yacc栈定义为int类型 */
    int yyerror(char *);   /* 不声明该函数将出现warning */
%}

%token TRUE_VAL FALSE_VAL
%token LPAREN RPAREN ENTER

```

```
%left  OR
%left  AND
%right NOT


%%

 /* 这样写prog可以让分析器每次读入一行进行分析，下一行重新分析expr */
prog : prog exprp
         | exprp
         ;

exprp   : expr ENTER {if($1) printf("true\n");
                        else printf("false\n");
                        }
                ;

expr  : expr OR expr              {$$ = $1 || $3;}
            | expr AND expr       {$$ = $1 && $3;}
            | NOT expr            {$$ = !$2 ;}
            | LPAREN expr RPAREN {$$ = $2;}
            | TRUE_VAL            {$$ = 1;}
            | FALSE_VAL           {$$ = 0;}
            ;


%%

int main(){
    yyparse();
    return 0;
}

```